Optimization and Parallelization of Pearson and Blur

Assignment: 2 - Performance Optimization and Tuning
Group: 4
Group members: Hazem Haj Ali and Obada Marawi

1. Introduction and Measurement Method

We investigated how source-level optimization and pthread parallelism affect two supplied C++ applications: pairwise Pearson correlation and a two-pass image blur. We first measured the original sequential programs, used profiling to select changes, and then parallelized the optimized code. We compared different input sizes and thread counts while checking that the outputs remained unchanged.

Pearson centers two vectors using their means and computes the dot product of their normalized deviations, producing one coefficient per unordered dataset pair. The supplied Blur is a separable weighted filter: it first filters horizontal neighbors and then vertical neighbors, normalizing by the weights of valid neighbors at image boundaries. We optimize these supplied computations rather than substitute a different image filter.

On the largest supplied inputs, we reduced median sequential execution time from 18.651 to 3.429 seconds for Pearson and from 11.959 to 3.818 seconds for Blur. In separate interleaved scaling experiments, we achieved 3.12x speedup for Pearson and 1.97x for Blur with 12 worker threads, compared with our optimized sequential versions measured in the same experiment. Both parallel programs passed the supplied verification scripts in our WSL2 environment.

1.1 Scope and environment

Item	Recorded configuration
Our platform	Ubuntu 22.04 under WSL2; Intel Core i7-1255U; 12 logical CPUs available
Compiler	g++ (exact local compiler version not preserved in the archive)
Build flags	-std=c++17 -g -Wunused -Wall; -pthread for parallel code; no -O flag
Profiling	Valgrind Callgrind 3.18.1; exclusive instruction counts
Inputs	Pearson: 128, 256, 512 and 1024.data; Blur: im1 to im4.ppm; radius 15

We ran our experiments under WSL2, which reports virtual CPU topology. We therefore do not interpret its topology as a reliable description of the host physical cores. The assignment evaluation machine uses a Xeon E5-2620 v4 with 8 physical and 16 logical cores. Our results describe our own environment and do not predict exact timings on that machine.

1.2 Measurement method

We recorded five runs per configuration. Elapsed values come from our Python monotonic-timing measurement series using time.perf_counter(), which Python documents as a clock for measuring durations [1]. GNU Time logs supplied user CPU seconds, system CPU seconds and peak resident memory using %U,%S,%M. We calculated CPU utilization as 100 x (user + system) / monotonic elapsed time. We report application-level runs, including file input/output, rather than isolated correlation or filtering kernels. The original timing driver is not archived, so we cannot independently reconstruct or subtract its wrapper startup overhead.

After we observed differences between GNU Time elapsed values and the monotonic clock, we asked our instructor and received approval to use Python for elapsed time. We also observed CPU-accounting anomalies in some sequential runs, so we treat CPU statistics as descriptive data rather than precise utilization measurements. We did not establish the cause of the discrepancies.

We use only the monotonic measurement series for the timing results in this report and exclude the earlier GNU-only timings. We compare medians while retaining all samples, and report the maximum peak memory across five runs. We kept compiler optimization settings unchanged and did not test -O2 or -O3. Most GCC optimizations are disabled without an optimization level [2].

We treated the supplied applications as batch workloads: the response metric is completion time for one fixed input. The course measurement lecture distinguishes workload characteristics, execution characteristics and resource use [7, PDF pp. 6, 13-15]. Smith and Williams emphasize representative, reproducible experiments and controlled workload, software and environment [14, pp. 208-212]. We retained the same supplied inputs and build flags, but our environmental controls were incomplete; we do not claim laboratory-grade reproducibility.

2. Baseline performance and bottlenecks

We measured the supplied sequential programs before changing the code. The table shows whole-process baseline measurements. We identify the supplied dataset and image files by their filenames so that the same inputs can be used for comparison throughout the experiments.

Program	Input	Median s	Min to max s	Peak MiB	Median CPU %
pearson	128	0.0531	0.0508 to 0.0604	4.14	76.6
pearson	256	0.3335	0.3307 to 0.3564	4.86	89.7
pearson	512	2.4139	2.3955 to 2.5346	8.74	93.5
pearson	1024	18.6507	18.5016 to 19.4991	23.79	93.9
blur	im1	0.5227	0.5154 to 0.5411	16.30	91.2
blur	im2	1.1135	1.1039 to 1.1424	24.64	92.7
blur	im3	1.9594	1.9222 to 2.0060	36.14	93.3
blur	im4	11.9589	10.1832 to 12.8089	132.68	93.8

We observed that Pearson baseline time increased sharply across the input labels. Blur also became slower and used more memory for the larger image labels. Our im4 baseline ranged from 10.183 to 12.809 seconds, so we interpret comparisons against its median with particular caution.

2.1 Callgrind evidence

We used Callgrind to record Ir, the number of executed instructions [3]. These percentages describe instruction work rather than elapsed-time shares. We profiled Pearson with 256.data and Blur with im1.ppm at radius 15. We measured native execution times separately from the instrumented runs.

Application	Original hotspot	Exclusive Ir share
Pearson	Vector copy constructor	38.01%
Pearson	Vector element access	17.61%
Pearson	dot	15.15%
Blur	Filter::blur	42.40%
Blur	Mutable r, g and b accessors combined	27.00%
Blur	Gauss::get_weights	7.26%

We used the Pearson profile to target redundant copies and temporary vectors. For Blur, we found weight generation inside both pixel loops: the annotation recorded 16,505,216 exponential-function calls for im1. We therefore moved this radius-dependent computation outside the loops. GCC documents loop-invariant motion and inlining as optimization techniques [2]; our changes apply related ideas explicitly at source level.

2.2 Baseline interpretation

Our baseline includes computation, allocation, file parsing and output formatting. We therefore use it to assess application-level performance rather than kernel-only speedup. CPU percentages near 100% are consistent with approximately one occupied logical CPU, subject to the accounting limitations. We checked output correctness separately instead of relying only on successful exit codes.

Data organization. Pearson uses a std::vector<Vector>; each Vector owns a contiguous double array. The parallel version adds one mean per dataset, an offsets array and a preallocated double result vector of N(N - 1)/2 entries. Blur stores R, G and B in separate row-major unsigned-char arrays, indexed as y * width + x. Its scratch channels instead use stride 3000. Worker task records hold pointers and ranges rather than private image or dataset copies; per-pair and per-pixel accumulators are local to each worker.

3. Pearson sequential optimization results

We measured the following cumulative Pearson versions incrementally while keeping compiler optimization settings unchanged. We preserved the arithmetic order within each pair calculation rather than splitting its floating-point reductions across threads.

The table identifies the cumulative changes. In the submitted source tree, the numerical changes are in pearson/analysis.cpp and the reference/accessor changes also affect analysis.hpp, vector.hpp and vector.cpp. The optimization_steps/V1 through V6 directories preserve the sequence; V6 contains the earlier changes, not a separate alternative to them.

Version	Change
V1	We used const references for datasets, pair inputs and dot arguments, with const-correct reading operators.
V2	We replaced four temporary vectors per pair with direct element calculations.
V3	We cached the vector length in the first loop only; this was an intermediate partial change.
V4	We used the cached length in both loops.
V5	We used const data pointers instead of repeated Vector operator[] calls in pair loops.
V6	We calculated each dataset mean once and reused it across pair comparisons.

Version	128 s	256 s	512 s	1024 s
Baseline	0.0531	0.3335	2.4139	18.6507
V1	0.0612	0.3422	2.3909	18.5011
V2	0.0351	0.1984	1.5829	10.9491
V3	0.0359	0.1901	1.3648	10.0202
V4	0.0259	0.1492	1.0080	7.4553
V5	0.0245	0.1346	0.8467	6.2370
V6	0.0184	0.0852	0.4675	3.4292

In V1, we reduced allocation and copy instruction work and peak memory, but did not establish a clear timing improvement. Larger-input gains were below 1%, while smaller medians increased. We therefore do not count V1 as a successful speed optimization in isolation. We measured clearer improvements with V2, V4, V5 and V6, while recognizing that separate experiment times limit causal attribution.

V1: avoid deep copies. Passing the dataset collection and Vector arguments by value invokes copying of owned arrays. We changed these parameters to const references and made reading operators const-correct. This follows the course case study on copying during argument passing [9, PDF pp. 8-9]. It also makes the no-mutation contract explicit. Reduced instruction work alone is not proof of a native elapsed-time improvement, as our V1 results demonstrate.

V2: remove temporary vectors. Originally, subtraction created two centered vectors and division created two normalized vectors for every pair. We replaced these four temporary arrays with two element loops: one accumulates squared deviations and the other normalizes each element and accumulates r. This removes repeated allocation, copying and destruction, addressing the excessive-dynamic-allocation pattern [13, PDF p. 34]. We retained separate square roots, divisions and accumulation order rather than using an algebraically rearranged formula that might round differently. The cost is more explicit numerical code.

V3 and V4: cache the loop bound. We moved vec1.get_size() into a local size value. V3 used it only in the first loop; V4 used it in both. The course explicitly covers caching loop-end values and moving invariant calls outside loops [8, PDF pp. 27, 32]. Here get_size() is already O(1): this change removes repeated call overhead in our no-O build, not an O(L) length search. An optimizing compiler may remove much of that overhead itself.

V5: bypass repeated accessors. We obtained const double pointers once and replaced vec1[i]/vec2[i] with x_data[i]/y_data[i] in the hot loops. The contiguous storage is unchanged; the intended saving is repeated out-of-line operator[] calls, not a new memory layout. Direct pointers expose representation and require correct lifetimes and bounds. This is especially relevant to our fixed no-O build; we have not established the same gain with compiler inlining enabled.

V6: reuse computed means. We fill a means array once before enumerating pairs and pass means[sample1] and means[sample2] to pearson. The course caching pattern recommends saving and reusing results instead of recomputing them [13, PDF p. 23]. This adds O(N) double storage and an O(N x L) preprocessing pass while removing repeated means from O(N squared) pair calls. Means remain valid because the datasets are not modified.

Input	Baseline to V6 speedup	Time reduction	Peak MiB before to after
128	2.89x	65.3%	4.14 to 4.08
256	3.91x	74.4%	4.86 to 4.64
512	5.16x	80.6%	8.74 to 7.64
1024	5.44x	81.6%	23.79 to 19.68

For 256.data, we reduced Ir from 3,985,190,203 to 717,373,555 (82.0%). Copy-constructor instructions fell from 1,514,752,361 to 2,628,073. The final profile attributed 63.22% to pearson itself, reflecting less surrounding work rather

than evidence that the function became slower. Appendix A gives the complete per-version instruction and resource summaries.

In V6, we added a small means array to avoid recalculating the same dataset means for every pair. For N datasets of length L, this reduces mean preprocessing from repeated O(N squared x L) work to O(N x L). Our pair calculations still require O(N squared x L) work overall.

4. Blur sequential optimization results

We evaluated two cumulative optimized Blur versions. We retained the horizontal pass followed by the vertical pass, the original boundary handling, and conversion of intermediate values to unsigned char. We kept this conversion because removing it could change output pixels.

Scratch is a temporary image buffer. The horizontal pass stores its completed pixel values there; the vertical pass reads those stable values to form the final image. Keeping input and output separate within each pass prevents later calculations from reading pixels that have already been overwritten. The scratch values remain bytes, as in the supplied algorithm.

4.1 V1 reuse Gaussian weights

In V1, we moved weight generation outside the pixel loops and calculated the weights once per call. We also cached image width and height. Since the weights do not depend on pixel position, we could reuse them without changing per-pixel arithmetic. Our original profile motivated this removal of loop-invariant work.

This is loop-invariant computation: for a fixed radius, Gauss::get_weights produces the same coefficients for every pixel. The course covers moving invariant work outside a loop [8, PDF p. 27], and the course text discusses loop-invariant code motion [15, Ch. 9]. Our weight generation changes from once per pixel per pass to once per blur call. The small weights array is shared read-only; filtering still uses the original neighbor order.

4.2 V2 direct access and row traversal

In V2, we replaced repeated r/g/b accessor calls in the filtering loops with direct channel pointers. We changed traversal to y first and x second, matching consecutive row storage. We kept the 3000 by 3000 scratch matrix and used its own stride of 3000 instead of the input image width. We also allocated radius + 1 weights to avoid the original array-size issue at radius 1000. All report measurements still used radius 15.

The loop order changes from x-outer/y-inner to y-outer/x-inner. Because a channel stores adjacent x coordinates consecutively, this gives consecutive output writes and row-wise traversal, consistent with the lecture on loop interchange [8, PDF p. 26] and cache efficiency [10, PDF pp. 5, 9]. Vertical neighbor reads still have a row stride: this is not a claim that every access becomes contiguous. We did not measure cache misses, so locality is the design rationale, not an isolated measured cause of the speedup. Both Blur changes are in blur/filters.cpp; mutable channel accessors are declared in matrix.hpp and defined in matrix.cpp.

Image	Baseline s	V1 s	V2 s	V2 speedup	Time reduction
im1	0.5227	0.3289	0.2067	2.53x	60.5%
im2	1.1135	0.7346	0.4395	2.53x	60.5%
im3	1.9594	1.3522	0.7585	2.58x	61.3%
im4	11.9589	6.2338	3.8180	3.13x	68.1%

Profile on im1	Executed instructions	Reduction from original
Original	6,838,922,864	0.0%
V1	4,186,708,177	38.8%
V2	2,432,455,953	64.4%

We measured a 37-44% reduction in native median execution time from Blur V1 to V2. Peak resident memory stayed nearly unchanged because we retained image copies and the full-size scratch allocation. Direct access and row traversal were introduced together, so we cannot separate their contributions. Appendix A reports CPU and memory monitoring for every sequential version, alongside baseline graphs.

Our changes removed the main repeated mutable accessor workload from the filtering loops, although accessor calls remain in copying and I/O-related paths. In the final profile, Filter::blur accounts for 73.38% of exclusive Ir because the direct-access calculations now take place inside that function.

4.3 Correctness and tradeoffs

We compared all four V1 and V2 output images with the original baseline and obtained byte-identical files. We could reorder independent pixels because the horizontal pass reads destination and writes scratch, while the vertical pass reads completed scratch and writes destination. Direct pointers reduce encapsulation, so we had to preserve correct bounds and strides.

5. Parallel implementation and synchronization

We used pthread_create and pthread_join in both applications and selected the requested worker count through a command-line argument. For one thread, the calling thread executes the task directly. With more than one, we create the requested workers and wait for them. Reading, writing and preprocessing remain sequential. Workers own disjoint output locations, so their result writes need no mutex. We join workers before shared task storage can be destroyed [4].

This is data decomposition over a shared address space. The course recommends independent, sufficiently large tasks, balanced loads and few synchronization points [11, PDF pp. 7-10]. Gerber et al. likewise distinguish task/data decomposition and explain why load imbalance and thread-management overhead limit performance [15, pp. 255-256]. We distribute coefficient pairs or output rows; we do not divide one coefficient's floating-point reduction among workers.

5.1 Pearson pair partitioning

For N datasets, we calculate N(N - 1)/2 unordered pairs. We represent the original nested-loop ordering with an offsets array, where offsets[i] marks the first result for pairs starting with dataset i. We allocate the result vector before starting workers so that they never call push_back concurrently.

For P requested threads and M pairs, we assign floor(M/P) pairs to each worker and one extra pair to each of the first M mod P workers. This balances pair counts more evenly than equal ranges of outer-loop indices. We use a binary search to locate each worker's starting pair, then advance through pairs in their original order.

Balancing pair counts matters because the triangular nested loop has N - i - 1 pairs in outer iteration i. Assigning equal numbers of outer iterations would give early workers more pairs. With equal-length supplied vectors, each pair has similar arithmetic cost, so flat pair intervals are a suitable static partition. A worker writes only its own preallocated result elements, while its accumulators and pair indices remain private.

We share datasets, means and offsets as read-only inputs. Each worker calls our sequential V6 pearson function for its assigned pairs and writes to its own result interval. We parallelize independent coefficients while keeping the reductions inside each coefficient serial. This preserves the floating-point summation order within each pair.

5.2 Blur row partitioning

For each Blur pass, we divide the H output rows into contiguous ranges. Worker i handles rows floor(H i/P) through floor(H (i + 1)/P) - 1. Workers share input channels and weights for reading, and write only to their own output rows.

We let the horizontal workers read destination and write scratch. We join all horizontal workers before creating the vertical workers because vertical neighborhoods may read scratch rows produced by other workers. The vertical workers then read scratch and write destination. We join them all before returning the result.

The join between passes is a dependency boundary, not an optional speed optimization. A vertical output may need scratch rows outside its own row interval, so starting it before all horizontal writes complete would be unsafe. We require no per-pixel mutex because each pass has read-only input and non-overlapping output ownership. This follows the course distinction between shared-memory access and synchronization ordering [11, PDF pp. 18, 31, 40-41].

We create a new set of workers for each pass. This keeps synchronization straightforward, but thread creation and two rounds of joins add overhead. A persistent worker team with a barrier between passes would be a possible improvement.

5.3 Interfaces and build

./pearson_par infile outfile num_threads
./blur_par radius infile outfile num_threads

We provided a Makefile in each program directory to build both sequential and parallel executables. We use -pthread when compiling parallel translation units and linking parallel programs. We successfully ran make clean followed by make for both applications in our group_4 submission folder. Existing signedness warnings in the supplied helper code did not prevent compilation.

5.4 Failure handling

We reject zero thread counts and invalid thread arguments. If thread creation fails, we join already-created workers before propagating an exception. We terminate the process on an unexpected join failure to avoid destroying shared task data while a worker might still use it.

6. Parallel timing and scalability

For each application, we ran a new experiment that interleaved sequential and parallel configurations across five recorded rounds per input. We tested 1, 2, 4, 8 and 12 threads, giving 120 runs per application. Interleaving reduces systematic ordering bias, although it does not eliminate background-load or thermal effects. We define S(P) as median optimized sequential time divided by median parallel time with P threads. We use the separate sequential executable as the reference, not the original unoptimized baseline.

6.1 Pearson

Input	Seq s	1 s	2 s	4 s	8 s	12 s
128	0.0183	0.0180	0.0163	0.0174	0.0152	0.0165
256	0.0840	0.0839	0.0630	0.0611	0.0547	0.0579
512	0.4954	0.4928	0.3180	0.2740	0.2415	0.2680
1024	3.3755	3.3728	1.9491	1.5530	1.2001	1.0829

6.2 Blur

Input	Seq s	1 s	2 s	4 s	8 s	12 s
im1	0.2099	0.2457	0.1559	0.1298	0.1126	0.1078
im2	0.4412	0.5185	0.3257	0.2787	0.2265	0.2208
im3	0.7659	0.9146	0.5774	0.4696	0.3978	0.3688
im4	3.9750	4.5972	3.0268	2.4800	2.1554	2.0132

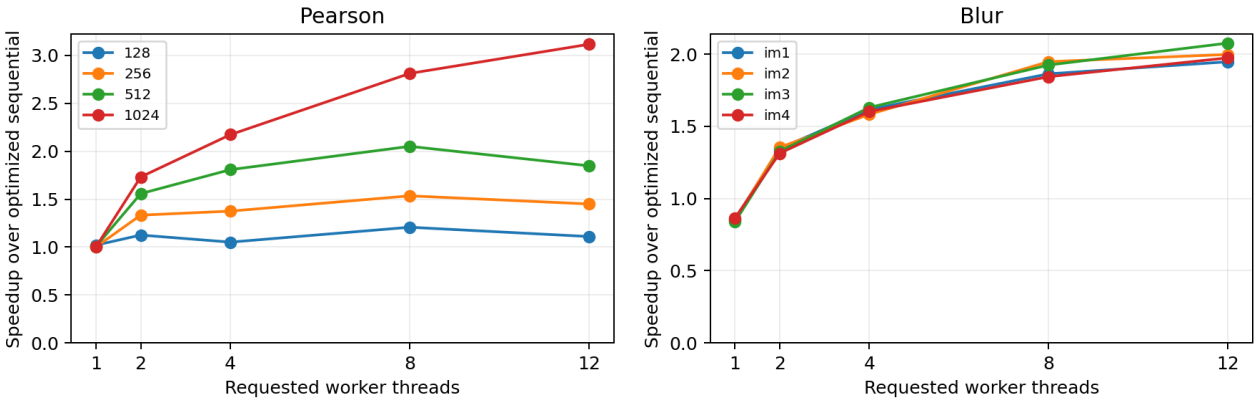


Figure 1. End-to-end speedup from the interleaved experiments. Lines connect measured thread counts; they are not predictions.

We measured 3.12x speedup for Pearson 1024 with 12 threads, while 8 threads gave lower medians for 128-512. For Blur, we measured 1.95-2.08x with 12 threads across the four images. These comparisons use optimized sequential code from the same experiment, not the original baseline. For Pearson 128, timing ranges overlap, so we cannot confidently rank the thread counts.

These are strong-scaling comparisons: for each curve we hold the input fixed and vary the requested worker count. We also calculate E(P) = S(P)/P, as in the multiprocessors lecture [12, PDF p. 23]. This is normalized speedup per requested worker, not a measurement of physical-core utilization. The largest-input values below make the scaling trade-off explicit.

Workers P	Pearson 1024 S(P)	E(P) %	Blur im4 S(P)	E(P) %
1	1.00x	100.1	0.86x	86.5
2	1.73x	86.6	1.31x	65.7
4	2.17x	54.3	1.60x	40.1
8	2.81x	35.2	1.84x	23.1
12	3.12x	26.0	1.97x	16.5

The 100.1% one-worker figure reflects a tiny timing difference between executables; it is not evidence of superlinear parallel scaling.

7. Resource use and correctness evidence

We compare optimized sequential measurements with the parallel configuration that had the lowest median time for each input. CPU percentages aggregate CPU time across workers. Values above 100% are expected during parallel work, but do not show that every requested CPU was fully occupied.

Program input	Best P	Seq peak MiB	Par peak MiB	Seq CPU %	Par CPU %
pearson 128	8	4.14	4.16	54.6	65.9
pearson 256	8	4.64	4.67	83.9	182.8
pearson 512	8	7.64	7.67	97.0	325.4
pearson 1024	12	19.67	19.83	99.5	588.5
blur im1	12	16.27	16.05	91.7	316.5
blur im2	12	24.65	23.89	97.0	362.0
blur im3	12	36.15	35.50	97.8	396.9
blur im4	12	132.62	132.56	99.8	368.1

We observed resident memory close to the sequential level. Peak RSS does not include every reserved virtual-memory byte, so we do not interpret it as total thread-stack allocation. Some of our shortest Pearson runs recorded 0.00 CPU seconds after rounding, making their CPU percentages unsuitable for precise utilization analysis.

7.1 CPU accounting caveat

In our earlier V6 sequential series, we observed CPU time greater than monotonic elapsed time in some runs: for example, 0.48 CPU seconds versus 0.463319 elapsed seconds for 512. We did not establish the cause. We keep the later interleaved measurements separate and interpret CPU statistics cautiously. Our performance conclusions rely on monotonic elapsed times and instruction counts.

7.2 Correctness checks

We obtained byte-identical baseline comparisons for every optimized sequential version on all four supplied inputs. We also checked parallel Pearson against baseline at 1, 2, 4, 8 and 12 threads. Its verify.sh passed at 2, 4, 8, 16 and 32 threads. For Blur, our byte comparisons passed at 1, 2, 4 and 8 threads, and verify.sh passed at 1, 2, 4, 8, 16 and 32 threads using the original baseline images as references.

After organizing the submission folder as group_4 and restoring its input directories, we rebuilt both applications from a clean state and reran the supplied verification scripts. Both returned exit code 0. These checks confirmed correct outputs for the supplied inputs and tested thread counts, but do not prove correctness for every possible input or the absence of all data races.

Our recorded monotonic series contain 440 measurements, all with exit code 0: 40 baseline runs, 160 incremental optimization runs and 240 interleaved scaling runs. We used exit status to check process termination and separate output comparisons to check numerical or byte equality. We did not perform a race-detector run or exhaustive malformed-input testing.

8. Discussion and conclusions

8.1 Interpretation and limitations

We improved Pearson by reducing copies, temporary vectors, accessor overhead and repeated means. We improved Blur by sharing weights and using direct row-based channel access. On the largest inputs, our incremental comparisons show 5.44x sequential Pearson speedup and 3.13x sequential Blur speedup over the original baselines. We measured these series at different times and did not quantitatively control temperature, background load or cache state.

For Blur, both V1 and V2 substantially reduced median time on every input, satisfying two measured optimization steps. For Pearson, V2, V4, V5 and V6 show substantial reductions on the larger inputs. V1 remains a documented unsuccessful timing attempt, and V3 is an intermediate partial edit, not an additional independent technique. This distinction matters because the assignment requires measurable improvements, not merely a list of edits.

We use the interleaved experiments for a stronger within-experiment parallel comparison. Sequential file handling, preprocessing and copying limit end-to-end scaling. Thread-management overhead and memory contention are additional possible limits, but we did not isolate their contributions. We collected no core-affinity, hardware-counter, frequency or temperature measurements and used five repetitions. WSL's 12 logical CPUs are not 12 equivalent dedicated physical cores.

Amdahl's law gives the ideal bound $S(P) = 1 / (s + (1 - s)/P)$ for serial fraction s under fixed-work assumptions [12, PDF p. 24]. Our I/O, copying, preprocessing and thread control remain serial, while real execution adds contention and

scheduling overhead. We therefore do not infer s from the measured speedup or expect 12x speedup from 12 requested workers. In particular, CPU utilization alone cannot identify the limiting resource.

The archive does not contain the original timing driver or a complete machine-configuration log. This limits independent reproduction of the exact timing procedure and platform details. A repeat should preserve the driver, compiler/Python versions, input dimensions or hashes and per-run environment settings. Five samples reveal variation but do not establish statistical significance; small differences and overlapping ranges are interpreted cautiously, consistent with the course warning about stochastic performance results [16, PDF p. 126].

We measured Blur with one thread as 16-19% slower than our separate optimized sequential executable. Both retain the same numerical operations, but the shared worker has extra direction choices and indexing. The one-thread path creates no worker threads, so thread creation cannot explain this gap. Extra worker-function overhead is a plausible explanation that we did not isolate experimentally.

8.2 Further improvements not implemented

We could further improve Pearson by precomputing magnitudes or normalized vectors. For Blur, we could reduce image copies, size scratch to the image, improve buffered I/O or reuse a persistent worker team. We would evaluate -O2 with a fresh baseline and correctness checks rather than assume the same gains. We did not implement the optional OpenMP version.

We would also test separate horizontal and vertical worker kernels, moving the invariant direction choice out of the innermost neighbor loop. The C/C++ lecture describes hoisting loop-invariant branches [8, PDF pp. 29-30]. This could reduce the one-thread Blur overhead, but it remains a hypothesis requiring fresh timing and correctness tests. Other candidates are a persistent worker pool and longer/batched runs for the shortest inputs; we would measure their gains rather than assume them.

8.3 Conclusion

We preserved the tested outputs while improving the sequential implementations and adding pthread parallelism. We found that the best thread count depends on input size. In our interleaved experiments on the largest inputs, 12 threads reduced Pearson from 3.376 to 1.083 seconds and Blur from 3.975 to 2.013 seconds relative to optimized sequential code measured in the same experiment.

9. References

[1] Python Software Foundation. Python 3.10 time documentation, perf_counter. https://docs.python.org/3.10/library/time.html

[2] GNU Project. GCC 11.4 Optimize Options. https://gcc.gnu.org/onlinedocs/gcc-11.4.0/gcc/Optimize-Options.html

[3] Valgrind Project. Callgrind user manual. https://valgrind.org/docs/manual/cl-manual.html

[4] Apple Developer Documentation Archive. POSIX-conforming pthread_join manual page. https://developer.apple.com/library/archive/documentation/System/Conceptual/ManPages_iPhoneOS/man3/pthread_join.3.html

[5] BTH. Assignment 2 - Performance Optimization and Tuning. Course assignment instructions supplied with this project, including requirements for baseline, iterative optimization, pthreads, correctness and submission.

[6] Our experimental records for Assignment 2. Packaged CSV files under measurements/baseline_monotonic, measurements/pearson_v1_monotonic through pearson_v6_monotonic, measurements/blur_v1_monotonic, measurements/blur_v2_monotonic, measurements/pearson_parallel_monotonic and measurements/blur_parallel_monotonic; corresponding text reports under profiling/pearson and profiling/blur.

Technical documentation checked 17 October 2026. Experimental values are calculated from our recorded CSV files [6].

[7] S. Memeti. Performance Measurements. DV1674 lecture slides, BTH, LP1 2023. Supplied file: 04_performance_measurements.pdf. PDF pp. 6, 13-15, 19, 43.

[8] S. Memeti. Optimization of C/C++ Programs. DV1674 lecture slides, BTH, LP1 2023. Supplied file: 05_optimization_of_C_programs.pdf. PDF pp. 26-32.

[9] S. Memeti. Optimization case studies: String use, Searching, and Sorting. DV1674 lecture slides, BTH, LP1 2023. Supplied file: 08_optimization_case_studies.pdf. PDF pp. 8-9.

[10] S. Memeti. Optimization of memory management and I/O. DV1674 lecture slides, BTH, LP1 2023. Supplied file: 07_memory_and_io.pdf. PDF pp. 5, 9.

[11] S. Memeti. Multithreading. DV1674 lecture slides, BTH, LP1 2023. Supplied file: 10_multithreading.pdf. PDF pp. 7-10, 18, 31, 40-41, 62-68.

[12] S. Memeti. Multiprocessors. DV1674 lecture slides, BTH, LP1 2023. Supplied file: 09_multiprocessors.pdf. PDF pp. 23-24.

[13] S. Memeti. Performance Patterns and Antipatterns. DV1674 lecture slides, BTH, LP1 2023. Supplied file: 12_performance_pattern_and_antipatterns.pdf. PDF pp. 21, 23, 34.

[14] C. U. Smith and L. G. Williams. Performance Solutions: A Practical Guide to Creating Responsive, Scalable Software. Addison-Wesley, 2002. Ch. 8, pp. 208-212. Supplied course extract.

[15] R. Gerber, A. J. C. Bik, K. B. Smith and X. Tian. The Software Optimization Cookbook: High-Performance Recipes for IA-32 Platforms, 2nd ed. Intel Press, 2006. Ch. 9, Loops, pp. 143-157; Ch. 15, pp. 255-256. Supplied course extracts.

[16] S. Memeti. Performance and Scalability testing. DV1674 lecture slides, BTH, LP1 2023. Supplied file: 03_performance_and_scalability_testing (1).pdf. PDF p. 126 (displayed slide 34/52).

Lecture citations use PDF page numbers because overlay pages sometimes differ from the displayed slide number. Textbook citations use printed page numbers. Course material supplies the rationale; only our own measurements support claims about achieved speedup.

Appendix A. Monitoring Across Optimization Steps

We report median CPU utilization (%) and maximum peak RSS (MiB) for each input at every cumulative sequential step. Each cell is CPU % / MiB. CPU values are descriptive because of rounding and the accounting anomalies discussed earlier. Ir is the total Callgrind instruction count in millions, profiled on Pearson 256 and Blur im1.

Pearson	128	256	512	1024	Ir million
Original	76.6 / 4.14	89.7 / 4.86	93.5 / 8.74	93.9 / 23.79	3985.2
V1	74.4 / 4.14	90.2 / 4.62	93.4 / 7.61	93.9 / 19.68	3108.3
V2	57.3 / 4.12	91.8 / 4.64	96.6 / 7.74	97.2 / 19.68	1752.9
V3	55.7 / 4.14	90.0 / 4.64	95.4 / 7.63	96.2 / 19.68	1652.6
V4	65.6 / 4.02	87.4 / 4.63	94.7 / 7.63	96.0 / 19.67	1551.9
V5	40.9 / 4.07	81.8 / 4.64	96.8 / 7.64	98.2 / 19.68	984.5
V6	51.8 / 4.08	88.1 / 4.64	102.7 / 7.64	98.9 / 19.68	717.4

Blur	im1	im2	im3	im4	Ir million
Original	91.2 / 16.30	92.7 / 24.64	93.3 / 36.14	93.8 / 132.68	6838.9
V1	96.3 / 16.27	99.1 / 24.64	99.1 / 36.14	99.8 / 132.71	4186.7
V2	97.5 / 16.29	97.8 / 24.66	97.7 / 36.19	99.5 / 132.59	2432.5

Pearson V1 reduced copy work and memory but did not establish a clear timing gain. Later steps reduced instruction work substantially; V6 also shows CPU-accounting anomalies. Blur peak RSS remains close to baseline despite large timing and instruction reductions. These resource measurements complement, rather than replace, the elapsed-time tables.

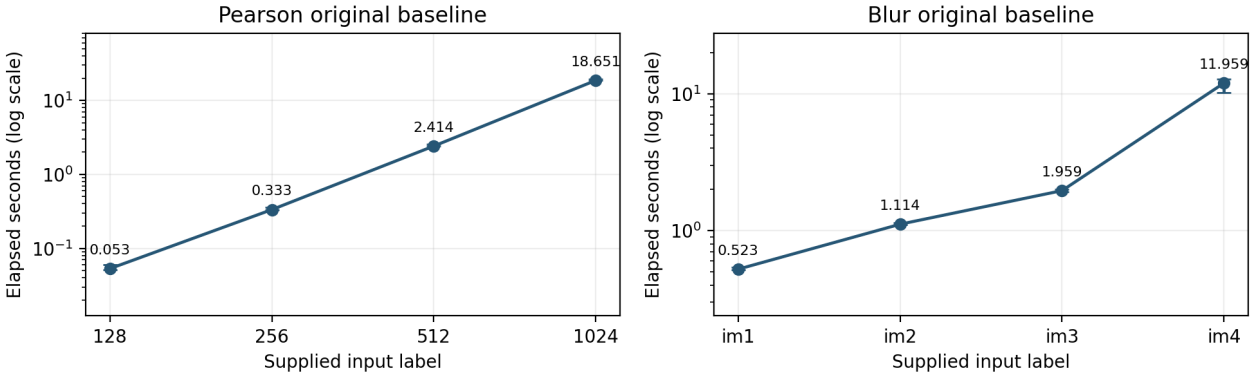


Figure 2. Original baseline median elapsed times; whiskers show the observed minimum and maximum of five runs. Each panel has a logarithmic time axis. The input labels do not imply equal increments in image size or workload.

We keep cumulative source versions under optimization_steps and record raw measurements separately. The organized V3 source snapshot was reconstructed from V4 and the confirmed partial edit; its CSV measurements remain the original recorded V3 runs. We do not count the reconstruction as a new performance experiment.

Assignment: *Performance Optimization and Tuning*

Group (4) members:
Hazem Haj Ali
Obada Marawi

Date: 2026-10-17